Plugfest Summary

Data Mapping partially tested, new Binding implementation started

Data Mapping

- Tested bitmap extraction for Sentron PAC
- Tested enum mapping for Echonet Lite Web API
- Found some “map” terms applicable for Ethernet IP Binding
- Pushed to node-wot branch “glc-plugfest”

Ethernet IP Binding for node-wot

- Coded based on the UA Edge Translator implementation
- Tested with Allen-Bradley PLC
- Also features browsing of PLC tags and TD creation based on it
- Pushed to node-wot branch “glc-binding-eip”

Not tested

- Could not test basic mathematical operations in Data Mapping
- Did not connect to LoRaWAN devices and test potential Data Mapping features there

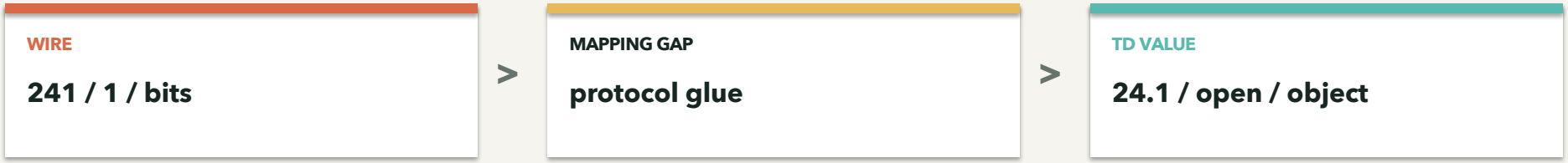
Other

- Implemented terminal explorer for TDs which could be consolidated and put along node-wot
- Got some hints for Domus TDD improvement, e.g., serialization to YAML-LD, CBOR-LD, or XML

Looking for devices to test remaining Data Mapping features

Data mapping in Thing Descriptions

Make protocol data usable as application data, without hiding the wire contract



This mismatch recurs across protocols; today the mapping often lives in binding-specific or application code

Demand: TD issues #2034 (Extending Data Mapping Examples), #2169 (Basic Operations on Data), #1936 (Supporting complex/structured types in simple protocols), #1930 (Supporting bitmaps)

Different symptoms, same mismatch

The wire representation is optimized for the protocol; the TD schema is optimized for the application

Scale and offset

Register 241 represents 24.1; a device may encode -5 as 5 plus an offset

Protocol codes

Wire value 1 may mean `open`; consumers need a stable semantic value, not a guessed label

Packed structures

Several registers or non-contiguous bits can represent one object with named fields

Tables and other media types

CSV/XML bytes can parse successfully while the column-to-DataSchema meaning remains unspecified

Recurring gap: content syntax and application meaning are separate contracts, but TDs need to connect them

Read and write mappings may differ

Mapping must account for direction, precision, and non-invertible choices

READ A FIXED-POINT VALUE

```
241  ->  24.1  ->  12.05
```

After an app change, inverse scaling yields 120.5; an integer register requires a rounding decision.

WRITE A CLASSIFIED VALUE

```
many wire values  ->  `low`
```

A many-to-one range has no unique reverse value unless a canonical representative is chosen.

Without explicit policies, consumers must guess or keep protocol-specific conversion logic

Demand: TD issues #2169 (Basic Operations on Data) and #875 (Should it be possible to indicate whether writing a property returns set value?); range-write policy in US4 analysis

The missing piece sits between decoding and validation

A consumer needs more than a media type and an application schema

DECODE

What bytes/text mean

contentType helps select a decoder; binary layouts and CSV row roles need binding context

>

MAP

Which value means what

Select fields, scale numbers, convert codes, or assemble records without out-of-band rules

>

VALIDATE

What the app receives

The TD DataSchema describes the final application-facing value and its constraints

Today these responsibilities are fragmented; equivalent mappings are repeated across bindings and consumers

Evidence: TD issues #1936 (Supporting complex/structured types in simple protocols), #1930 (Supporting bitmaps), #2169 (Basic Operations on Data); node-wot PR #1125 (Support Objects in octet-stream codec)

Useful pieces exist, but solve different parts

The question is not whether a TD can mention CSV or enums; it is whether a consumer can execute the mapping portably

DataSchema / JSON Schema

Types, constraints, and enum domains describe the application value
They do not generally transform a different wire value into it

Forms and `contentType`

`contentType: text/csv` can select a CSV decoder
It does not define row classification or column-to-DataSchema mapping

Protocol-specific vocabularies

BACnet value maps, PROFINET payload layouts, and LoRaWAN transforms encode valuable but binding-local semantics

Custom codecs and consumer code

Can handle device-specific cases today, but the rules may be external to the TD and duplicated by consumers

Setting `contentType` is valid today; the remaining gap is the declared, interoperable mapping to the DataSchema

Examples: BACnet / PROFINET / LoRaWAN binding specifications; OpenAPI XML PR #4592 (How to represent XML elements with attributes)

Related terms already exist, but live at different layers

The protocol surveys reveal useful precedents and the assumptions hidden in each vocabulary

LORAWAN

```
lorav:multiplier
lorav:offset
lorav:divisor
lorav:transform
```

Value transforms; close to generic numeric steps

BACNET

```
bacv:hasValueMap
bacv:hasProtocolVal
bacv:hasLogicalVal
```

Enum semantics; binding-specific vocabulary

PROFINET

```
profv:byteOffset
profv:bitOffset
profv:payloadMapping
profv:enumeratedValue
```

Wire layout + structure + enum decoding

MODBUS

```
Wire data types
No arithmetic map terms
in reviewed binding
```

Raw numeric decode is distinct from scaling

Finding: reuse common value semantics; keep byte layout and payload-tree navigation with their format/protocol bindings

Source: US3-details.md; US4-details.md; US5-details.md; US2345-general.md

The missing middle is the mapping contract

Across numeric, enum, structural, side-channel, and non-JSON cases, the same interoperability gap appears



A portable TD should tell a consumer how decoded protocol data becomes the declared application value, and how writes travel back when that mapping is reversible

Not a separate solution per user story: one missing contract appears in many concrete forms

Demand sources: TD issues #2034 (Extending Data Mapping Examples), #2169 (Basic Operations on Data), #1936 (Supporting complex/structured types in simple protocols), #1930 (Supporting bitmaps), #997 (Modeling enumeration values semantically)

Let’s revisit the problem through one shared mapping model

How a TD can describe the transformations

NUMERIC

241 -> 24.1

Scale on read; make inverse rounding explicit

ENUM

1 -> open

Map protocol codes to application values

STRUCTURE

reading.v -> 23.7

Select the field, then transform the decoded value

CSV

AirTemp_C -> 23.4

Decode rows, pick a column, coerce, validate

One mapping contract
Content-type bindings decode formats; ordered steps connect values to the TD DataSchema

Decode, map, validate

Keep wire-format decoding distinct from the transformation into the TD-facing DataSchema




```
"map:fromWire": [ ... ]
"map:toWire":   [ ... ]
```

Read and write are explicit, ordered directions
If a reverse mapping is lossy or ambiguous, the TD must say how writes are handled

Proposed shared pattern in US3-details.md, US4-details.md, US5-details.md, and US2345-general.md

The same pipeline composes different transformations

Operations are ordered; each step consumes the previous result

```
"map:valueMapping": {
  "map:fromWire": [
    { "map:proc": "pick", ... },
    { "map:proc": "mul", "map:value": 0.1 },
    { "map:proc": "enum", ... },
    { "map:proc": "record", ... }
  ]
}
```

Scalar transform

241 -> 24.1

Code to meaning

1 -> `open`

Structure to schema

nested field / CSV rows -> TD-shaped value

Format-specific selectors run at the binding boundary; format-neutral operations share the same mapping contract

Examples grounded in issues #2169, #1936, #1930 and the node-wot CSV pilot

A readable value may not be the wire value

Common fixed-point encodings trade a simple device representation for a useful app value

SCALE

121 **x 0.2** 24.2

Raw register 121 represents 24.2 in the application

OFFSET

5 **- 10** -5

An offset can expose signed meaning over an unsigned wire range

As a TD designer, describe the transform so consumers need not embed device-specific arithmetic

Source: US3-details.md; GitHub issue w3c/wot-thing-description#2169

Direction and order are part of the meaning

Small declarative steps avoid hiding transformations in consumer code

```
"map:valueMapping": {
  "map:fromWire": [
    { "map:proc": "mul", "map:value": 0.2 }
  ],
  "map:toWire": [
    { "map:proc": "mul", "map:value": 5 },
    { "map:proc": "round", "map:mode": "ceil" }
  ]
}
```

Example:
121 -> x 0.2 -> application value 24.2
24.1 -> x 5 = 120.5 -> round to wire value 121

- **Candidate primitives**

  mul / add / reciprocal / round / clamp

- **Read and write are explicit**

  fromWire and toWire can differ when precision is lost

- **Quantization is visible**

  Rounding is a declared step; it is not an accidental side effect

Source: US3-details.md; issue #2169 fixed-point precision discussion

What if fixed primitives are not enough?

Color temperature is reciprocal: the app exposes kelvin; the wire value is mirek

APP VALUE

Kelvin

1,000,000 / x

WIRE VALUE

Mirek

Each direction also clamps and rounds to the device's supported range

QUESTION

Fixed operation catalog, or a bounded expression language with input placeholders?

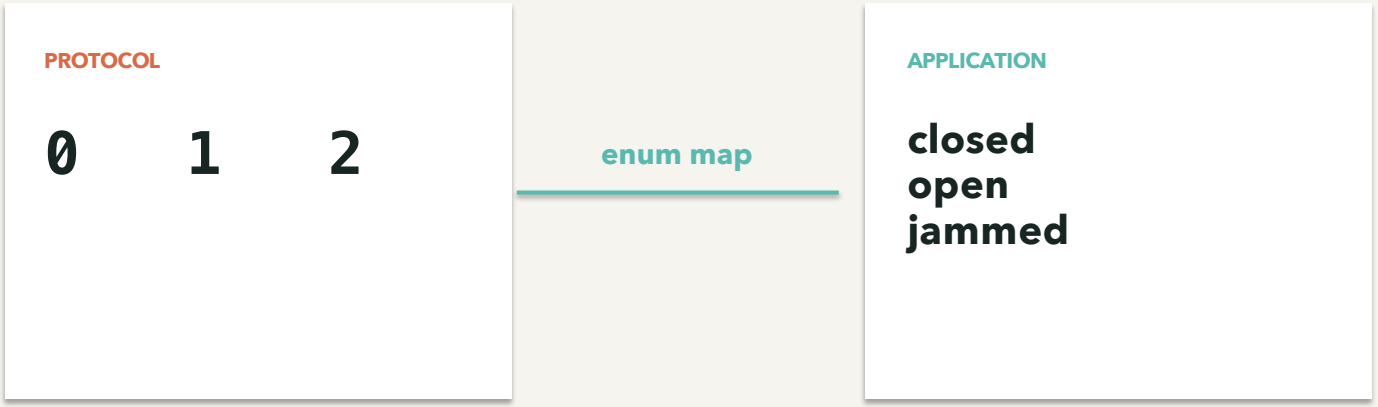
```
”expressionLanguage": "xpath-3.1",
"map:fromWire": {
  "expression": "let $mirek := min((500,
    max((153, $input)))) return min((6536,
    max((2000, round-half-to-even(1000000
    div $mirek)))))"
},
"map:toWire": { "expression": "..." }
```

An expression is not automatically safe: the issue rejects arbitrary executable code and calls for a constrained DSL

Source: issue #2169 comment 5996673540; US3-details.md

A code can have a better application name

Enum conversion changes the value domain, not just its label



- **Exact map**

  Unambiguous wire/app pairs.

- **Range map**

  Non-overlapping intervals; writes need a canonical wire value.

Keep both directions explicit: a many-to-one or lossy mapping cannot be safely inverted by guessing

Source: US4-details.md; BACnet and PROFINET enum examples

One Modbus register, six application states

Siemens OpenAir Override control: an unsigned holding-register code maps to a semantic string

APPLICATION DATASCHEMA

```
"type": "string",
"enum": [
  "off", "open",
  "close", "stop",
  "goToMin", "goToMax"
]
```

App-facing values

```
"href": "modbus+tcp://gateway.example:502/1/1",
"contentType": "application/octet-stream",
"modv:entity": "HoldingRegister",
"modv:type": "xsd:unsignedShort",
"op": ["readproperty", "writeproperty"],
"map:valueMapping": { "map:fromWire": [{
  "map:proc": "enum",
  "map:map": [
    {"map:wire":0,"map:app":"off"},
    {"map:wire":1,"map:app":"open"},
    {"map:wire":2,"map:app":"close"},
    {"map:wire":3,"map:app":"stop"},
    {"map:wire":4,"map:app":"goToMin"},
    {"map:wire":5,"map:app":"goToMax"}
  ]
}] }
```

Exact lookup: preserve semantic states for the application; keep integer codes on the wire

Payload shape can differ from the DataSchema

The application wants a stable model even when the protocol wraps, packs, or batches values

```
{ "device": {
    "reading": { "v": 237 }
  }
}
```

pick / unwrap

TD-FACING VALUE

23.7 C

- **Select or reshape**
  pick, place, wrap, unwrap, at, setAt
- **Convert packed fields**
  bitExtract / bitCompose for decoded integers
- **Compose steps**
  Combine structure, scaling, and enum mapping

Source: US5-details.md; structural operators are candidates, not a final standard

One Sentron PAC4200 value carries 17 boolean flags

The Modbus value spans two registers; the flags occupy non-contiguous bytes and bit positions

```
"href": "modbus://pac4200.local/holding-register/203",
"contentType": "application/octet-stream",
"map:valueMapping": { "map:fromWire": [{
  "map:proc": "bitExtract",
  "octv:fields": [
    {"octv:fieldName":"limit0",
     "octv:mask":16777216,"octv:shift":24,
     "octv:fieldType":"boolean"},
    {"octv:fieldName":"limit8",
     "octv:mask":65536,"octv:shift":16,
     "octv:fieldType":"boolean"},
    {"octv:fieldName":"logicFlag",
     "octv:mask":1,"octv:shift":0,
     "octv:fieldType":"boolean"}
  ]
}] }
```

1 | PROTOCOL DECODE

Modbus selects the two-register value at offset 203.

2 | OCTET / BIT LAYER

octv masks and shifts extract fields from the decoded word.

3 | TD DATASCHEMA

The application sees one object with 17 named booleans.

Boundary: byte/bit layout is binding-specific; the TD schema names the application fields. Excerpt shows 3 of 17 masks.

CSV parsing is not the same as data mapping

A CSV row is text with positional or header-based meaning; the TD needs named, typed values

```
"TOA5", "CR1000", ...
"TIMESTAMP","RECORD","AirTemp_C","RH","WindSpeed_ms"
"TS","RN","Deg C","%","m/s"
"","","Avg","Avg","Avg"
"2026-06-25 10:00:00",1,23.4,56.2,4.8
"2026-06-25 10:01:00",2,23.5,56.0,4.6
```

- **csvv payload binding**

  Dialect, encoding, header and metadata rows, and row representation

- **map core**

  Select a decoded value, coerce its type, or project every row into records

Today: declare contentType: text/csv
Open gap: describe how CSV values map to the DataSchema

Source: US2-details.md; node-wot examples/datamapping/toa5-weather.csv

One cell, decoded and validated end to end

TOA5 weather fixture: object rows, explicit metadata filtering, then pick + coerce

```
"contentType": "text/csv",
"csvv:rowRepresentation": "object",
"csvv:headerRow": 1,
"csvv:metadataRows": [0, 2, 3],
"map:fromWire": [
  { "map:proc": "pick",
    "jsonv:path": "/0/AirTemp_C" },
  { "map:proc": "coerce",
    "map:type": "number" }
]
```

CSV	**metadata rows filtered**
PICK	**"23.4"**
COERCE	**23.4**

DataSchema validation runs before value() returns

Source: node-wot examples/datamapping/README.md and toa5-weather.td.json

One mapping can project a whole time series

The CSV codec returns all decoded rows; generic record mapping shapes every row for the TD

```
"map:proc": "record",
"map:fields": {
  "timestamp": [
    { "map:proc": "pick",
      "jsonv:path": "/TIMESTAMP" },
    { "map:proc": "coerce",
      "map:type": "string" } ],
  "value": [
    { "map:proc": "pick",
      "jsonv:path": "/WindSpeed_ms" },
    { "map:proc": "coerce",
      "map:type": "number" } ]
}
```

TD-FACING RESULT

```
[
  {"timestamp":"2026-06-25 10:00:00",
"value":4.8},
  {"timestamp":"2026-06-25 10:01:00",
"value":4.6}
]
```

Prototype boundary

CSV read/decode + pick/coerce/record + schema validation are implemented

Not yet implemented: CSV serialization or map:toWire
Prototype semantics are not normative

Source: node-wot examples/datamapping/toa5-weather.td.json and README.md

A small core with content-type bindings

The decoded representation determines which binding terms apply; the core stays format-neutral

CORE

map context

valueMapping
fromWire / toWire
proc + ordered steps
scalar ops + record

jsonv

JSON tree
path / template

octv

byte layout
bit fields

csvv

dialect / rows
headers / metadata

Binding decodes wire representation; core mapping transforms decoded values

Working namespaces from US2345-general.md; not normative TD vocabulary